Note: There are 3 projects of different lengths; the first one is far shorter.
The labs use redpitaya board with a daughter board with switches, a button and a 7-segment display. The appendices give some information about simulation using ghdl and gtkwave; synthesis using in particular yosys and programming using openFPGALoader. The end of each projects will use Vivado to configure a processor and have a link between the Processing System (PS) on linux and the Programmable Logic (PL).
You only need to install on your computer ghdl and gtkwave to work at home (since you do not have the board).
In the lab PCs, use as a working directory: home/etudiant/ELISE2/your_name.
Put useful comments in your VHDL files (no description but the purpose). In the report, you have to **explain** your choices, all the things you learned and why the simulations show it works correctly. Attach to the report all the .vhd files (including the testbenches) **only them**. Do not forget to simulate all components before assembling them.
Reports are due to before the 3rd lab for the 1st project, before the 5th lab for the 2nd one, and 2 weeks after the last lab for the 3rd one.
The use of ChatGPT or any other AI is strictly prohibited.

PROJECT 1: DDS

In order to better understand the working operation of the AD 9834 DDS that you will communicate with in the 3rd project, you will synthesize a DDS on a FPGA. Simply put, a DDS is composed of 3 parts: the **phase accumulator** (therefore an adder), an offset generator (another adder) and some memory containing the shape of the desired signal.

The DDS output frequency is:

$$f_{out} = f_{MCLK} * W/2^N$$

where f_{MCLK} is the master clock of the DDS (here the FPGA 125 MHz) and N the size in bits of the word W which adjust the output frequency. A DDS uses the fact that the phase is intrinsically linear: at each time increment Δt, the phase increase with the same value (modulo 2π) linked to the frequency: $\Delta\varphi = 2\pi f \Delta t$. If the phase is coded in N bits (2^N corresponding to 2π), then W corresponds to the phase increase. The phase accumulator is then designed as an adder, which increases the coded phase with a W value. The memory (here RAM) allows the link between a phase and the corresponding amplitude of the signal according to its shape.

For this project, N will be a parameter (GENERIC before PORT) with a default value of N = 10. The signal shape will be a sinus, quantified with 14 bits (unsigned: 0 to 2**14-1). The inputs will therefore be W, the phase offset (10 bits) and the clock. The output is a 14-bits signal.

1) Use octave software to compute the signal shape stocked in the memory and that you will copy/paste into your .vhd file. Write your vhdl description of the DDS.

Write the test bench and simulate. In your simulations with gtkwave, you can choose the format “analog” to see the “shape” of your output.

Study the result with different value of W. What happens if $W > 2^9$? Explain with a graph in timing and another one in frequency. What is the largest value of the output frequency f_{out} ? What is the purpose of the offset, which adds some value to the phase obtained from the phase accumulator? In particular, what is its obvious purpose when there are several DDS ?

2) Before programming the board, you need to write an encapsulating .vhd file. Check the ADC_DAC_channel1.vhd example to know how to use the 125 MHz DAC on the red pitya board. You will set the offset to 0 and set 2 bits of W directly in the encapsulating file,

the others will be linked to the switches (do not forget to put the jumpers of the board for the GPIO pins).
Before using the makefile makefile.openXC7, **source /home/jmfriedt/openxc7/export.sh**
Test by visualizing on an oscilloscope several values of W. You can modify if you want the clock in the encapsulating file to slow down the signal.

3) For the last part of this lab, you will aim to log into the linux on the red pitaya and directly change W in order to have all the possibilities for W and the offset, and not be limited by the number of switches. To configure the interface between PS and PL you will use Vivado software from Xilinx.

To launch Vivado:
type « export LANG=en_US.UTF-8 »
then « source /home/jmfriedt/Xilinx/Vivado/2024.2/settings64.sh »
finally « vivado & »

Open *TP_M2_source* project in home/etudiant/seb and save it (from vivado) into your folder. This project includes the processing system, a port (AXI_GPIO) to link PS and PL and the converters management.

Note: if a window is hidden, you can obtain it with the menu « Window » at the top. Except for *Project Manager*, to change « settings » of the various grouping in the « Flow Navigator» (to the left), you have to right click on the grouping.

Open the block design (Flow Navigator → IP Integrator). You can undock the *Diagram* window with the *Float* icon at the top right. Right-clicking in this window enables you to create ports (*Create Port*), to add modules that you have already designed (*Add Module*) and to add modules from proprietary libraries (*Add IP*). Study the global operation of this project.

Add your DDS.vhd file (without the encapsulation) in your project with File → Add sources or by clicking on the « Add Sources » icon in the « Project Manager » part of the « Flow Navigator» (top left). In the diagram window, add your DDS.vhd file (add Module). If you modify the source file, you need to refresh the module (right click on it and select refresh modue). You can change the names of the instances by selecting them, then with the *Properties* window (under the *Sources* window).

Double click on the GPIO component and select “enable Dual Channel” to have 2 output ports. Change the GPIOs width to 10. Click OK to confirm the choice and close the window. Click on the AXI bus « + » of the GPIO component to see all the inputs/outputs. You can connect the outputs gpio_io_o to your DDS inputs by click and drag. The s_axis_tdata of the DAC component needs 32 bits. You can add the *constant* IP and the *concat* IP to fill the MSb with zeros. Connect a constant ‘1’ to the input s_axis_tvalid. Modify the axis_red_pitaya_dac.vhdl:

Replace “int_dat_a_reg <= int_dat_a_wire(DAC_DATA_WIDTH-1) & (not int_dat_a_wire(DAC_DATA_WIDTH-2 downto 0));”
with “int_dat_a_reg<= int_dat_a_wire(DAC_DATA_WIDTH-1 downto 0);”
Explain the reason of this change knowing the output of the DAC is $\pm$1V. Do not forget to refresh the module. Save the project.
Click on *validate design* icon (check icon)to check problems (the warning about the clock for the ADC is ok).

Generate the bitstream (from the *Flow Navigator*), all the previous necessary synthesis steps will be automatically performed. This bitstream creation can take a lot of time, so you should continue.

Another way to program the FPGA is directly from the linux on the chip, without using a JTAG probe. In your project folder on the computer, write with a text editor the .bif file for the creation of the .bit.bin file:

all:
{
project_path/ project_name.runs/impl_1/bistream_name.bit
}

You will be able to use bootgen to create the .bit.bin file when the .bit file is finally made:
/home/jmfriedt/Xilinx/Vivado/2024.2/bin/bootgen –image bif_file.bif –arch zynq –process_bitstream bin

With a terminal, connect to the red pitaya board via ssh (after plugging everything…):
ssh root@192.168.2.200
(pwd root)
Within another terminal, copy the .bit.bin file into the red pitaya folder /tmp (if the .bit.bin is not yet generated, use in the meantime the one in /home/etudiant/seb/docs):
scp -O fichier_a_copier root@192.168.2.200:/tmp
With the red pitaya prompt, copy the .bit.bin file into /lib/firmware
Note that you can also communicate with the red pitaya board usining minicom with the following parameters /dev/ttyUSB0, 115200 8N1, no flux correction (minicom –s to change the configuration) and use the USB as a serial communication instead of ethernet.
From the red pitaya board, flash the FPGA:
echo nom_du_bitstream.bit.bin > /sys/class/fpga_manager/fpga0/firmware
For your information, fpga_manager homogeneize the management of FPGA within linux and replace the former Xilinx FPGA configuration using xdevcfg.

With vivado, when the *Diagram* window is displayed, you can read the GPIO memory space in the Address Editor (should be 0x4120_0000).
AXI_GPIO datasheet tells the following offsets:
0: GPIO_DATA
4: GPIO_TRI (1 → input, 0 → output)
8: GPIO_DATA2
0xC: GPIO_TRI2
0x11c: GIER
0x128: IP IER
0x120: IP ISR
From the red pitaya, you can read the data at address 0x41200000 (for instance) with:
devmem 0x41200000 w
(w = word = 32 bits, you can also use h = 16 bits and b = 8 bits)
To write XXXX into the address 0x41200000: devmem 0x41200000 h XXXX
Modify GPIO_TRI and GPIO_TRI2 to assign the 2 GPIO as outputs.
Modify GPIO_DATA and GPIO_DATA2 to change W and the offset. Check the effect with an oscilloscpe. You need to use a 50 Ω load to avoid bouncing.

PROJECT 2: FIR

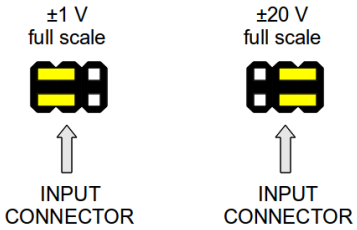
In this lab, you will design a FIR (Finite Impulse Response) filter working at 125/8 = 15.625 MHz in order to reduce the number of stages and having a cutoff frequency in the MHz range. At the end, you will also use again Vivado for the interaction between PL and PS.

1) With octave software, use *fir1* function (from *signal* package) to compute the coefficients of a fourth order low-pass filter, with a -6 dB cutoff frequency fc = 0.5 MHz and a sampling rate fs = 125/8 MHz.
b = fir1(4, fc/(fs/2), 'low')
Use *freqz* function to visualize Bode plot: freqz(b)
What is the real -6 dB cutoff frequency? Explain why it is not 0.5 MHz.

2) Write the filter description using 8 bits for the coefficients (see lecture for the structure and to use integer and not floating point coefficients) and 14 bits (unsigned) for I/O. The coefficients will added as parameters (GENERIC) and an input enable will also be added.

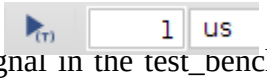
Write the test bench and simulate. **Why in our case do we not loose information with a 14-bit output? (Why do not we need more bits than 14?)**

Check the example file ADC_DAC.vhd. Write an encapsulating .vhd file. During the compilation, if you get an error message about SITEWIRE/DSP48_X1Y30/CARRYCASCIN, ask your favorite FPGA teacher.
Test that there is no hardware problem with one of the example programs with a function generator as the input (use 0.5V peak-peak amplitude (-0.25V to 0.25V)) **check with an oscilloscope before plugging!**) and an oscilloscope.
Then test your program and check the normal behavior of your low-pass filter.
Note: with the LV (Low Voltage) configuration, maximal input volatge is ±1V, with HV (High Voltage) configuration it is ±20V.



3) In this part, you will create a band-pass filter using the FIR Compiler IP in Vivado.
With octave, still with *fir1* function, compute the coefficients of a 30th order band-pass FIR with cutoff frequencies of 2 and 2.5 MHz and a sampling rate fe = 125/8 MHz.
Once again, check the real cutoff frequencies.
In a Terminal, type « export LANG=en_US.UTF-8 »
then « source /home/jmfriedt/Xilinx/Vivado/2024.2/settings64.sh »
finally « vivado & »
Create a new Vivado project: File → Project → New
Give a name to the project and its working folder (home/etudiant/ELISE2/your_name/project_name)
Choose RTL project.
At the bottom of the « Add Sources » window, put « VHDL » for « Target language » and « Mixte » for « Simulator language ». The source files will be added later.
Add the constraints file ports.xdc.
As the red pitaya board is not a default board, in the « Parts » tab choose the FPGA chip: xc7z010clg400-1 (Zynq-7000 family)

Create a block design (Flow Navigator → IP Integrator).
Add the *FIR Compiler* IP. Double click to update the following parameters:
filter option: coefficient vector to directly fill with floating numbers
channel spec: input sampling frequency and clock frequency at 15.625 MHz
implementation: coeff width (8), tick "best precision fraction length", manual input data type (signed), manual input data width (14), output rounding mode (Full Precision)
Add the ports and link them to the module. For the s_axis_data_tvalid input of the FIR, use the *Constant* IP to permanently set it to '1'.
Click on *validate design* (icon in *Diagram*) to verify there is no major error.
In the *source* tab, right-click on the name of the design and choose *Create HDL Wrapper* (let Vivado manage wrapper and auto-update).

Create a test bench (.vhd simulation file) with « Add Sources ». The component you need to use in the test bench is the wrapper.
Notice than most of the syntax errors are automatically underlined in red. Also notice the light bulb shaped icon for templates.
By saving the file instantiating the entity, « Simulations Sources » hierarchy is automatically updated if there is no problem. Launch the simulation from the « Navigator Flow ». Notice that if syntax error is not underlined in the text window: you need to look into the « Messages » tab in the bottom window. Saving the test bench .vhd file updates these messages. For some errors, as indicated in the messages, you have to look into the xvhdl.log file.
Click then on « Run Simulation » in the « Flow Navigator » (behavioral simulation)
Remember that a window can be undocked with the « float » icon at its top right.
In the test_bench_behav.wcfg window, the icon will zoo fit your simulation.
In the main window, you can click on the run time icon (will the amount of time next to it) to continue your simulation. 

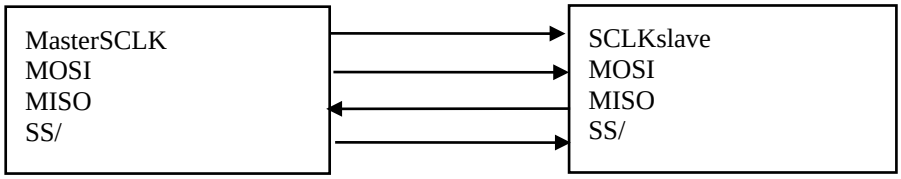
By right-clicking on a signal in the test_bench window, you can change the radix. You can also change the waveform style (and choose analog if needed).
Verify the proper behavior of your description. Careful of the latency of the FIR (see the summary tab when double-clicking on the FIR Compiler IP)
If we want a 14-bit output, which bits should we keep? What will happen to the gain? Discuss it in relation to the previous low-pass filter.

4) Open *TP_M2_source* project in home/etudiant/seb and save it (from vivado) into your folder. This project includes the processing system, a port (AXI_GPIO) to link PS and PL and the converters management.
Add your 2 filters and create a .vhd source file in order to manage them: depending on the least significant bit of the AXI_GPIO port output, the 2 filters outputs are inverted on the DAC ('0' to have the low-pass filter on the DACa (16 LSBs of the AXI bus) and pass-band filter on the DACb (16 MSBs of the AXI bus); with '1' the DACa corresponds to the pass-band filter and DACb corresponds to the low-pass filter). Use channel 1 (A) for the input of the filters. In the 32-bit bus for the entities axis_red_pitaya_adc and axis_red_pitaya_dac, the most significant 16 bits are for channel 2 (B) and least significant 16 bits are for channel 1 (A). For theses two 16 bits, the 2 MSb duplicate the sign bit (bit 13). You can use the IP *slice* to use only a part of a bus. Add a new PLL clock (clk_wiz) : f_{clk3}=fs
Like the first project (DDS), generate the bitstream (from the *Flow Navigator*). Either use openFPGALoader to load the .bit or load the .bit.bin from the red pitya (after writing the .bif file and creating the .bit.bin file with bootgen).
Modify the GPIO registers with devmem.

With the same care as before, test your project with a function generator and an oscilloscope. In particular, check the cutoff frequencies. Comment the look of your curves at high frequencies.

PROJECT 3: SPI

We wish to have a SPI (Serial Peripheral Interface) communication with the DDS (Direct Digital Synthesizer) AD9834. The SPI bus, developed by Motorola, is synchronous, full-duplex and has 4 wires: SCLK (clock), MISO (Master In, Slave Out), MOSI (Master Out, Slave In) and SS/ (slave select).



Here the master is the FPGA and the slave the DDS. There will be a communication only from the master to the slave, in order to configure the DDS. In the datasheet of the AD9834 chip, SDATA corresponds to MOSI and FSYNC to SS/.

The SCLK period, t_4, must be 25 ns at least (40 MHz max). For our VHDL description, the prescaler (FPGA clock division to obtain SCLK frequency) is obtained with the *tscale* parameter (>2) with a default value of 20 (f_{SCLK} = 6.25 MHz by default).

DDS communication is performed with 16-bit packets with MSb fist. As the DDS read data during SCLK falling edges, you will change SDATA during SCLK rising edges.

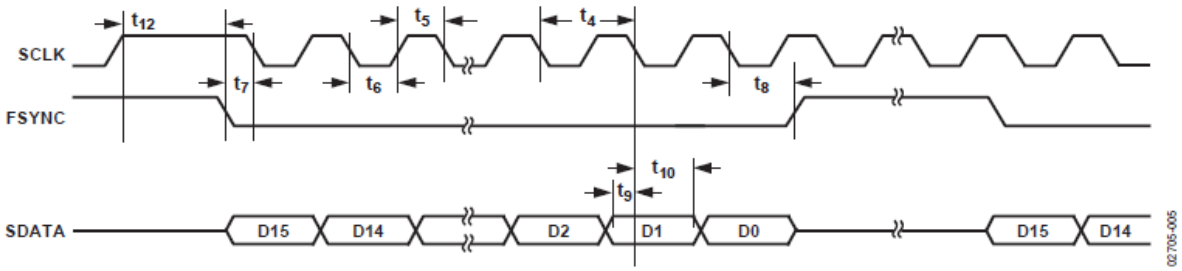


Figure 5. Serial Timing

The DDS datasheet advises to have the RESET input at 1 (active) during initialization then put at 0 when the configuration is done and the chip is ready to send the signal.

The output frequency is defined by:

$$f_{out} = f_{MCLK} * W/2^N$$

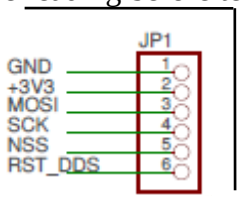
where f_{MCLK} is the master clock (here the 70 MHz clock generated by the µC), N (here 28) the number of bits coding the phase increment W. You need therefore 28 bits to program W and send 2 packets of 16 bits.

The control register can be programmed when the 2 MSb (D15 and D14) equal 0. D9 equaling 1 means that the reset function is managed with the hardware pin and not through software. D13 indicates if the whole 28 bits (D13=1) will be updated or only 14 bits (chosen with D12). When D13=1 (the configuration we will use), we send first the LSB then the MSB. Two frequencies (actually phase increments) can be configured and memorized: we will use FREQ0, which is configured when D15=0 and D14=1 (D13 to D0 being the configuration of the 14 LSB with the first packet and the 14 MSB with the 2nd packet).

1) You will use a hierarchic conception in 3 steps:

- An entity managing SCLK with a *tscale* prescaler (parameter by default equaling 20) and sending N bits (parameter by default equaling 16) according the previous time diagram. An output will go to 1 when all N bits have finished transmitting. An input enable/reset allows (when 1) the sending or reset (when 0) of the clock (SCLK=1), the frequency divider counter and the bit counter. **In order to have the change of MOSI occurring at the same time than an edge of SCLK, both signals will be managed with the frequency divider counter.**
- A finite state machine (Moore machine) managing the communication, in particular the previous entity and SS/=FSYNC. Pressing a button will lead to FYNC at 0 during 1 FPGA clock period, then will authorize the previous entity to send the N bits (FSYNC still at 0). When all the N bits are sent and the button is released, the machine will go back to its default state waiting for a new button pressing. Depending on the previous entity, intermediate states may have to be added. **Only one sequence of N bits is sent with one button pressing** (sent when button pressed, not released)**.**
- An entity taking into account the resources of the board. Two switches will choose between 4 possibilities of 16-bit words to send. The others 2 will serve to choose between the 4 half-byte of the word to display on the seven-segment display. The 8 leds will also display the corresponding byte. The Button will be used to send the word. A reset output (not included in the Moore machine) will be added and fixed: the DDS datasheet say inactive corresponds to 0. Outputs with the GPIO will be as follow (continue reading before testing):
  - SDATA : GPIO_[0]
  - SCLK : GPIO_[1]
  - FSYNC : GPIO_[2]
  - RESET : GPIO_[3]

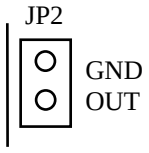
DDS board



Do not forget to give the state diagram in the report (with de transition conditions and the outputs for all the states). The first entity will of course be simulated and tested with an oscilloscope before working on the Moore machine. In the same way, the Moore machine will be simulated before being incorporated.

The STM32 on the DDS board must generate the 70 MHz clock and have its MOSI and SCLCK pins put at high impedance. To do that, program it with chenillard_v2 (on moodle): moving LEDs will indicate the successful writing.

First send 0x2200 to the DDS for the configuration, then 2 packets (LSB then MSB) to choose the frequency. You need to send again 2 packets to have another frequency. To change which frequencies are possible you can modify the 3rd entity.



Output on the STM32/DDS

2) Open *TP_M2_source* project in home/etudiant/seb and save it (from vivado) into your folder. Modify it to include your SPI project with the 16-bit word chosen with the GPIO.
Like the other projects, generate the bitstream (from the *Flow Navigator*), program the FPGA and modify the GPIO registers with devmem.

Test with an oscilloscope and conclude.

APPENDICES 1: Simulation with open tools

As a habit, use the same name for the entity and the .vdh file.
To analyze and see potential errors, type in a terminal:
ghdl -a nameofthefile1.vhd nameofthefile2.vhd
In the case of the use of a "work" package, put the package file first.
In order to use standard 2008 VHDL (which allows /* */ for several lines commentaries for instance):
gdhl -a --std=08 nameofthefile.vhd
If there is no error, the prompt will display nothing but the file work-obj93.cf (or work-obj08.cf for VHDL 2008) will be created.
After writing the test_bench (see example in appendices 3), you can check to combination of the two in the same way (and see potential errors in the test_bench):
ghdl -a testbench.vhd nameofthefile.vhd
To create the simulation file: ghdl -r entity_test_bench options --wave=name_simu_file.ghw
for instance: ghdl -r simu --stop-time=100ns --wave=wave.ghw
Note that you need to use ghdl -r --std=08 if you used ghdl -a --std=08
Launch gtkwave with: gtkwave name_simu_file.ghw
Click on the entities or signals in the top left window to make signals appear in the *signals window* (just bellow the previous window). Then click in this *signal window* on the signals you want to visualize in the *wave window*.

APPENDICES 2: Programming with open tools

Names of the inputs and outputs declared in the entity should correspond to the names in the ports.xdc file (case sensitive). Even if you use only one bit of a vector (one switch for instance), use a 2-bit std_logic_vector. In case of using the 125 MHz redpitaya clock you need to use the components IBUFDS and BUFG (refer to appendices 4 of M1 lab subject or filADC_DAC_channel1.vhd, no need to use special files for these components). To comply with these two requirements without modifying the .vhdl file you simulated you can create another .vdh file to encapsulate it (using component/Port Map).
Type once **source /home/jmfriedt/openxc7/export.sh**
Modify the makefile that you can find in moodle (makefile.openXC7).
PRJ corresponds to the name of the project but also the name of the .vdh file with the top level entity, which also must be the name of this top level entity.
VHDL_BIBLI corresponds to all the other .vdh files needed (can be left blank if there is no other .vhd file)
V_LIST should contain project_name.v (verilog file which will be created from the .vhd files). It could also contain other verilog files but that won't be our case.
CONSTR contains the constrain files, in our case ports.xdc
When typing *make -f makefile.openXC7*, you may have on some PC the error "ImportError: cannot import name 'antlr_to_tuple'" but it still should work (the size of the .bit file should be several Mo).
yosys with the ghdl plugging will convert the VHDL .vdh files into a verilog .v file.
yosys will then make the RTL (Register Transfer Level) synthesis step.
nextpnr will make the place and route steps. Only some FPGA are supported with this non-proprietary tool.
Thanks to Project X-ray (fasm2frames, xc7frames2bit) the bit-stream with the right format for Xilinx XC7 FPGAs can be created (.bit).
make load will use openFPGALoader to program the FPGA thanks to the JTAG H2S programming cable from Digilent. The JTAG connector is situated near the Ethernet plug of the redpitaya with the names of the pins (3V3, GND, TCK, TD0, TDI, TMS) written under the board.
Do not forget to perform *make clean* before performing *make.*

APPENDICES 3: Example of testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity simu is
--  Port ( );
end simu;

architecture Behavioral of simu is
        component entity_to_simulate is
    Port ( clk :  in STD_LOGIC;
                    SW : in STD_LOGIC_VECTOR(3 downto 0);
                    Button : in STD_LOGIC;
          led : out STD_LOGIC_VECTOR(7 downto 0) );
end component;
        signal clkt: STD_LOGIC;
        signal SWt : STD_LOGIC_VECTOR(3 downto 0);
        signal buttont : STD_LOGIC;
        signal ledt : STD_LOGIC_VECTOR (7 downto 0);
        constant PERIOD : time := 8 ns; -- 125 MHz

        begin
                tb: entity_to_simulate PORT MAP(clk=>clkt, SW => SWt, Button=> buttont,
led => ledt);

                clk_process: process
                begin
                   clkt <= '0';
                   wait for PERIOD/2;
                   clkt <= '1';
                   wait for PERIOD/2;
                end process;

                button_proc: process
                begin
                        buttont <= '0';
                   wait for 8 ns;
                   buttont <= '1';
                   wait for 10*PERIOD;
                end process;

                SW_proc: process
                begin
                        FOR i IN 0 TO 15 LOOP
                                SWt <= std_logic_vector(to_unsigned(i,4));
                                wait for PERIOD;
                        END LOOP;
                end process;
end Behavioral;
```